Sprint 1 - Agent Enrollment, Agent Spawning and Frontend

End of sprint: an agent can be enrolled through a defined workflow, the system spawns the right agents from a user's brief, each agent's data is stored in dynamically created, schema-validated tables, and users can do all of this from the frontend.

US_ID	Feature	User Story	Acceptance Criteria	Story Points
US_01	**Agent Enrollment**	As a platform engineer, I want to enroll a new agent through one defined workflow so that the orchestrator can discover and use it without code changes.	- Enrollment takes name, description, capabilities, supported models, input/output schema and cost profile. - An enrolled agent appears in the registry with a unique ID and an Active status. - Missing required fields and duplicate agent names are rejected with a clear error. - Enrollment completes in a single request and in under 3 seconds. - An agent can be updated or disabled; disabled agents are never offered to the orchestrator.	**8**
US_02	**Agent Spawning**	As a user, I want the system to read my brief and spawn the right agents, more than one when needed, so that complex requests are handled by a coordinated team.	- The system maps the brief to required capabilities and spawns one or more enrolled agents. - Only the agents needed for the brief are spawned; the rest are skipped with a reason. - Each spawned agent gets its own instance ID and status. - If no enrolled agent matches, nothing is spawned and the user is told which capability is missing. - Empty or vague briefs are rejected or answered with a clarifying question.	**13**
US_03	**Dynamic Table Creation**	As a data analyst, I want a table created automatically for each agent from its output schema so that structured results are stored without manual database changes.	- A table is created when an agent is enrolled, with columns and types taken from its output schema. - Table names are generated safely and unique; unsafe names or SQL in names are rejected or sanitised. - Running creation twice does not produce an error or a duplicate table. - If table creation fails, enrollment is rolled back and no half-created agent remains.	**8**
US_04	**Schema Validation**	As a user, I want agent schemas and outputs validated so that bad data never reaches storage or the next step.	- A malformed schema is rejected at enrollment with an error naming the problem field. - Agent output is validated against the schema before it is written to the table. - Output with a missing field or wrong data type is rejected, the step is marked Failed and nothing is stored. - Validation errors are logged with agent ID and field.	**5**
US_05	**Frontend**	As a user, I want a workspace where I can submit a brief, see enrolled agents and watch spawned agents so that I can use the platform without calling the API.	- The /office page has a brief input box and submit button; empty briefs are blocked. - An agents page lists enrolled agents with capabilities and status, matching the API. - After a brief is submitted, spawned agents appear with their status and selection reason. - Validation and error messages from the backend are shown clearly to the user.	**8**